An-Najah National University

Normalization Document

Virtual Hospital Management System

Course	Database
Instructor	Dr. Firas Shakaa
Team Members	Yaqeen Sofian Hasan (12325842) Rahaf Azzam Lahwani (12340170)

Introduction:

This document explains the normalization process for our Virtual Hospital Management System database. The purpose of normalization in our project is to organize the hospital data in a clean way, avoid repeated data, and make sure the tables are easy to update and manage.

Our system stores information about patients, doctors, departments, appointments, visits, diagnoses, treatments, prescriptions, medications, laboratory tests, bills, payments, rooms, and admissions. Instead of keeping all this information in one large table, we divided the data into separate related tables. This makes the database more organized and reduces mistakes when adding, updating, or deleting records.

Why Normalization Was Needed:

If all hospital information was stored in one table, the same data would repeat many times. For example, a patient name and phone number could repeat with every appointment, visit, bill, and admission. A doctor name and department information could also repeat with every patient visit. This would make the database harder to maintain.

For that reason, we separated the data into smaller tables. Each table has one main purpose. For example, patient information is stored in the Patient table, doctor information is stored in the Doctor table, and billing information is stored in the Bill and Payment tables.

Starting Relation Before Normalization:

At the beginning, the hospital data could be imagined as one large relation like this:

```
Hospital_Record(patient_id, patient_name, gender, date_of_birth, phone, address, medical_history,
doctor_id, doctor_name, specialty, department_id, department_name, appointment_id,
appointment_date, visit_id, diagnosis_id, treatment_id, prescription_id, medication_id, test_id,
bill_id, payment_id, room_id, admission_id)
```

This relation is not a good design because it mixes different types of data together. It includes patient data, doctor data, appointment data, medical data, billing data, and room data in the same place. This causes repetition and makes the system more likely to have update errors.

Functional Dependencies:

Functional dependencies show which attributes are determined by a key. In our project, each main table has a primary key that identifies the full row. The main functional dependencies are shown below:

Determinant	Determined Attributes
department_id	department_name, location
doctor_id	full_name, specialty, phone, email, department_id
patient_id	full_name, gender, date_of_birth, phone, address, medical_history
appointment_id	appointment_date, appointment_time, status, patient_id, doctor_id
visit_id	visit_date, notes, appointment_id, patient_id, doctor_id
diagnosis_id	diagnosis_name, description, visit_id
treatment_id	treatment_name, description, cost, visit_id
prescription_id	prescription_date, instruction, visit_id
medication_id	medication_name, dosage_form, price
test_id	test_name, test_date, status, visit_id
result_id	result_details, result_date, test_id
bill_id	bill_date, total_amount, status, patient_id
payment_id	payment_date, amount_paid, payment_method, bill_id
room_id	room_number, room_type, status
admission_id	admission_date, discharge_date, patient_id, room_id
prescription_id, medication_id	one row in Prescription_Medication

Primary Keys:

Each table in the final design has a primary key. The primary key is used to identify each record uniquely. The only table with a composite primary key is Prescription_Medication because it represents the many-to-many relationship between prescriptions and medications.

Table	Primary Key
Department	department_id
Doctor	doctor_id
Patient	patient_id
Appointment	appointment_id
Visit	visit_id
Diagnosis	diagnosis_id
Treatment	treatment_id
Prescription	prescription_id
Medication	medication_id
Prescription_Medication	prescription_id, medication_id
Laboratory_Test	test_id
Lab_Result	result_id
Bill	bill_id
Payment	payment_id
Room	room_id
Admission	admission_id

First Normal Form (1NF):

The database is in First Normal Form because every attribute contains one value only. There are no repeated groups inside a single field. For example, the Patient table stores one phone number, one date of birth, one address, and one medical history value for each patient record.

The many-to-many relationship between prescriptions and medications was also handled correctly. We did not store multiple medications inside one prescription field. Instead, we created a separate table: Prescription_Medication(prescription_id, medication_id)

This keeps the values atomic and makes the design cleaner. All tables are in 1NF because all values are atomic and there are no repeating groups.

Second Normal Form (2NF):

The database is in Second Normal Form because it is already in 1NF and every non-key attribute depends on the full primary key. Most tables have a single primary key, so partial dependency cannot happen in those tables.

Examples:

Patient(patient_id, full_name, gender, date_of_birth, phone, address, medical_history)

Doctor(doctor_id, full_name, specialty, phone, email, department_id)

Bill(bill_id, bill_date, total_amount, status, patient_id)

In these tables, the non-key attributes depend on the whole primary key. The table Prescription_Medication has a composite key, but it does not have extra non-key attributes, so there is no partial dependency. All tables are in 2NF because there are no partial dependencies.

Third Normal Form (3NF):

The database is in Third Normal Form because it is in 2NF and there are no transitive dependencies. This means non-key attributes do not depend on other non-key attributes.

For example, if we stored doctor and department data together in one table, we would have this problem:

```
Doctor_Department(doctor_id, doctor_name, specialty, department_id, department_name, location)
```

In this case, doctor_id determines department_id, and department_id determines department_name and location. This creates a transitive dependency. To avoid that, we separated the design into two tables:

```
Department(department_id, department_name, location)

Doctor(doctor_id, full_name, specialty, phone, email, department_id)
```

Another example is billing. If bill and payment data were stored in one table, the bill information would repeat for each payment. So we separated them into:

```
Bill(bill_id, bill_date, total_amount, status, patient_id)

Payment(payment_id, payment_date, amount_paid, payment_method, bill_id)
```

All tables are in 3NF because every non-key attribute depends only on the key of its table.

BCNF Check:

The final schema also satisfies BCNF. In our tables, the left side of each important dependency is a primary key or a superkey. For example:

```
patient_id -> full_name, gender, date_of_birth, phone, address, medical_history

doctor_id -> full_name, specialty, phone, email, department_id

appointment_id -> appointment_date, appointment_time, status, patient_id, doctor_id

bill_id -> bill_date, total_amount, status, patient_id
```

Since the determinants are keys, the relations are well normalized. The Prescription_Medication table also has no non-key attributes, so it does not create a BCNF problem. The project schema satisfies BCNF because every determinant is a key or superkey.

Dependency Preservation:

The decomposition is lossless because the tables can be joined back together using primary keys and foreign keys without losing information. For example, Doctor and Department are connected by department_id. Appointment is connected to Patient and Doctor by patient_id and doctor_id. Bill and Payment are connected by bill_id.

Example:

```
Doctor.department_id references Department.department_id

Appointment.patient_id references Patient.patient_id

Appointment.doctor_id references Doctor.doctor_id

Payment.bill_id references Bill.bill_id
```

Because of these relationships, the system can join the tables when needed and still keep the data separated in a clean way.

Dependency Preservation:

The design preserves the main dependencies because each table keeps the attributes that depend on its primary key. This means the database can check the main rules inside each table without needing to combine many tables every time.

For example, patient details are checked inside the Patient table, doctor details are checked inside the Doctor table, and bill details are checked inside the Bill table. This makes the design easier to understand and easier to implement in SQL and Java.

Final Normalized Schema:

After normalization, the final database schema became:

Relation	Attributes
Department	department_id, department_name, location
Doctor	doctor_id, full_name, specialty, phone, email, department_id
Patient	patient_id, full_name, gender, date_of_birth, phone, address, medical_history
Appointment	appointment_id, appointment_date, appointment_time, status, patient_id, doctor_id
Visit	visit_id, visit_date, notes, appointment_id, patient_id, doctor_id
Diagnosis	diagnosis_id, diagnosis_name, description, visit_id
Treatment	treatment_id, treatment_name, description, cost, visit_id
Prescription	prescription_id, prescription_date, instruction, visit_id
Medication	medication_id, medication_name, dosage_form, price
Prescription_Medication	prescription_id, medication_id
Laboratory_Test	test_id, test_name, test_date, status, visit_id
Lab_Result	result_id, result_details, result_date, test_id
Bill	bill_id, bill_date, total_amount, status, patient_id
Payment	payment_id, payment_date, amount_paid, payment_method, bill_id
Room	room_id, room_number, room_type, status
Admission	admission_id, admission_date, discharge_date, patient_id, room_id

Conclusion:

The Virtual Hospital Management System database is normalized and organized into separate tables based on the type of data. The design reduces repeated data, avoids update problems, and keeps relationships clear using primary keys and foreign keys.

The schema satisfies 1NF because all values are atomic. It satisfies 2NF because there are no partial dependencies. It satisfies 3NF because there are no transitive dependencies. It also satisfies BCNF because every determinant is a key or superkey. Overall. Therefore, the final schema is well-structured and suitable for implementing the hospital